

PAINEL ADMINISTRATIVO

O que foi implementado, e o que ficou diferente

Documento	Relatório de conformidade do Painel Administrativo
Área	Backend / Gestão de Sistemas
Baseado em	Especificação Técnica: Configuração do Painel Administrativo e Dados de Teste — Luiz Eduardo Andrade de Oliveira
Contrato vigente	Contrato de Integração da API — Versão 4.0
Código analisado	<code>backend/prototipo_backend/EasyRide/</code>
Branch	<code>feat/grupos-filtros-admin</code>
Testes	146 testes, todos passando

1. Objetivo

Este documento compara o back-end com a Especificação Técnica do Painel Administrativo, item por item.

Para cada requisito, mostramos onde ele foi implementado e qual teste o cobre. Quando o código ficou diferente do documento, explicamos por quê. A intenção é que a mesa de homologação encontre a resposta aqui, sem precisar abrir o código para descobrir.

2. Antes de tudo: qual contrato vale

A Especificação do Painel foi escrita quando o contrato da API estava na Versão 2.0. Desde então o contrato avançou duas vezes, e a Versão 2.0 foi removida do repositório no commit `ecc8ffc` :

Versão 2.0 (citada no item 9.2 da Especificação) → **Versão 3.0** → **Versão 4.0** (vigente, `docs/Atualizacao_Contrato_API.pdf`)

Daí vem uma coisa importante para a homologação: **o item 9.2 pede para conferir a massa de dados contra a Versão 2.0, e isso não é mais possível** — seria preciso voltar o back-end para um contrato que a própria equipe já substituiu. A conferência foi feita contra a Versão 4.0, que é a que vale hoje, e os valores batem: `perfil` (paciente, familiar, cuidador), `origem` (google, instagram, facebook, indicacao, evento, outro) e `tipo_instituicao` (clinica, hospital, ong, outro).

Isso também explica boa parte do que vem a seguir: três das diferenças listadas (D2, D3 e D4) vieram dessa mudança de contrato, e não de escolhas desta entrega. As outras duas (D5 e D6) são decisões de código, explicadas nas seções 5.5 e 5.6.

3. Resumo geral

Todos os itens da Especificação foram atendidos. Cinco pontos ficaram diferentes do texto do documento, e um sexto vem de uma decisão tomada antes dele, na revisão do DER.

Nada aqui espera decisão de ninguém. A seção 5 explica cada caso: por que ficou assim e como o objetivo do requisito foi cumprido de outra forma.

Situação	Requisito	Observação
Atendido	Item 2 · Dados de PF e PJ na mesma tela	Com proteção extra contra cadastro misto PF+PJ
Atendido	Item 3 · Tela de listagem, filtros e busca	As 7 colunas pedidas, mais 4 extras; 25 por página, navegação por data, busca por CPF/CNPJ
Atendido	Item 4 · Exportar para CSV	Colunas, separador e acentuação conforme pedido
Atendido	Item 5 · Permissões por grupo	3 grupos; Vendas só altera o status do lead
Atendido	Item 6 · Cenário 1 (filtrar empresas)	Filtro lateral de tipo de pessoa
Atendido	Item 7 · Testes CT01 a CT03	Uso por teclado, 100+ registros e busca com caracteres especiais
Atendido	Item 8 · Dados de teste	Os 15 cadastros do documento (9 pessoas / 6 empresas)
Atendido	Item 9.1 · Nomes dos campos na API	<code>dados_fisica</code> e <code>dados_juridica</code> existem na API (<code>serializers.py:48-49</code>)
Atendido	Item 9.2 · Massa de dados x contrato	Conferida contra a Versão 4.0, vigente; ver seção 2
Divergente	D1 · Cenário 2 do item 6 (<code>save_model</code>)	Tratado em outros dois lugares; ver 5.1
Divergente	D2 · Nomes dos status do lead	O sistema tem mais status que o documento; ver 5.2
Divergente	D3 · CPFs e CNPJs de teste	Começo do número mantido, dígito final corrigido; ver 5.3
Decisão de DER	D4 · Coluna <code>tipo_pessoa</code> no banco	Não existe, por decisão registrada na revisão do DER; ver 5.4

Situação	Requisito	Observação
Divergente	D5 · Nomes usados na busca (itens 3.3 e 9.1)	O documento usa os nomes da API num lugar que exige os nomes do banco; ver 5.5
Divergente	D6 · Acentuação do CSV (item 4)	Mesmo resultado no Excel, escrito de outro jeito; ver 5.6

4. Requisitos atendidos

Item	O que o documento pede	Onde está no código
2.1	Dados de PF e PJ embutidos na tela de Pessoa	<code>admin.py:42-57</code> , <code>admin.py:101</code>
2.2	Telas de Interesse, Pessoa e Modelo	<code>admin.py:178</code> , <code>:92</code> , <code>:169</code>
3.1	7 colunas, 25 por página, status editável na própria lista	<code>admin.py:186-204</code> — ver 4.1
3.2	Filtros laterais e navegação por data	<code>admin.py:191-203</code>
3.3	Busca por nome, e-mail, telefone, CPF e CNPJ	<code>admin.py:197-200</code> — ver 5.5
3.4	Carregar os dados relacionados de uma vez só	<code>admin.py:246-259</code>
4	Exportar CSV: colunas, separador <code>;</code> e acentuação no Excel	<code>admin.py:304-365</code>
5	Os 3 grupos de permissão	<code>permissoes.py</code> , <code>migrations/0007_grupos_rbac.py</code> , <code>signals.py:35</code>
6	Cenário 1 — filtrar por pessoa física ou empresa	<code>admin.py:60-80</code>
7	CT01 (teclado), CT02 (100+ registros) e CT03 (busca)	<code>seed.py --bulk</code> , <code>Ct02PaginacaoTest</code> , <code>InteresseAdminTriagemTest</code>
8	Os 15 cadastros do documento (9 pessoas / 6 empresas)	<code>management/commands/seed.py:77-336</code>

4.1. O que foi feito além do pedido

Quatro coisas foram feitas além do que o documento pede, porque sem elas o painel teria problemas na prática:

- **Quatro colunas a mais na listagem** — o item 3.1 pede 7 colunas, e a tela tem 11 (`admin.py:186-190`). As 7 pedidas estão todas lá; foram somadas `origem` , `aceite_termos` , `possui_cadeira` e o e-mail da pessoa. São os dados que o operador usaria para decidir o que fazer com o lead — sem eles, seria preciso abrir registro por registro só para ver de onde o contato veio.

- **Esconder o formulário errado** — `get_inline_instances` (`admin.py:103`). Uma Pessoa pode ser Física ou Jurídica, mas o banco não impede que ela seja as duas ao mesmo tempo. Sem esse cuidado, abrir uma clínica no painel mostra também o bloco de Pessoa Física em branco, convidando o operador a criar um cadastro misto que a exportação não sabe representar.
- **Uma consulta em vez de uma por linha** — `select_related` (`admin.py:257`). O documento pede para carregar Pessoa e Modelo de uma vez só, mas o Django já faz isso sozinho. O ganho de verdade está em carregar junto os dados de Pessoa Física e Jurídica, que o Django não traz por conta própria. Medimos: 11 consultas ao banco para exibir 10 leads, contra 1 depois do ajuste.
- **Vendas edita só o status** — `get_readonly_fields` (`admin.py:223`). A matriz do item 5 diz que Vendas pode alterar apenas o `status_lead` . Só que a permissão do Django é tudo ou nada: quem pode editar um lead, pode editar qualquer campo dele. Travar campo por campo exigiu código próprio.

5. O que ficou diferente, e por quê

5.1. D1 — O `save_model` do Cenário 2 (item 6)

O que a Especificação pede. Ao cadastrar um lead pelo botão "Adicionar Interesse" com um e-mail que já existe, o sistema deve bloquear a gravação **dentro do método** `save_model` e mostrar a mensagem "Lead já cadastrado com este e-mail (Conflito 409)".

O que existe. Esse método não foi criado. Mas o problema que ele resolveria — e-mail repetido causando erro 500 ou lead duplicado — está tratado nos dois lugares onde alguém realmente digita um e-mail no sistema.

Por que esse método não faria nada. Na tela de Adicionar Interesse, a Pessoa não é digitada: ela é escolhida numa lista de pessoas que já estão cadastradas. Não existe campo de e-mail nesse formulário.

Ou seja, quando o sistema chega ao `save_model`, a pessoa selecionada já está no banco — e o e-mail dela já é único, porque o campo tem essa regra desde o começo (`models.py:29`).

Uma verificação de duplicidade ali seria uma condição que nunca acontece. E o teste dela só passaria se forçássemos, na mão, uma situação que a tela não produz — dando aparência de cobertura sobre código que ninguém executa.

O cenário do documento descreve uma tela que cadastra a pessoa a partir de um e-mail digitado. Essa tela existe no projeto, mas não é a de Adicionar Interesse:

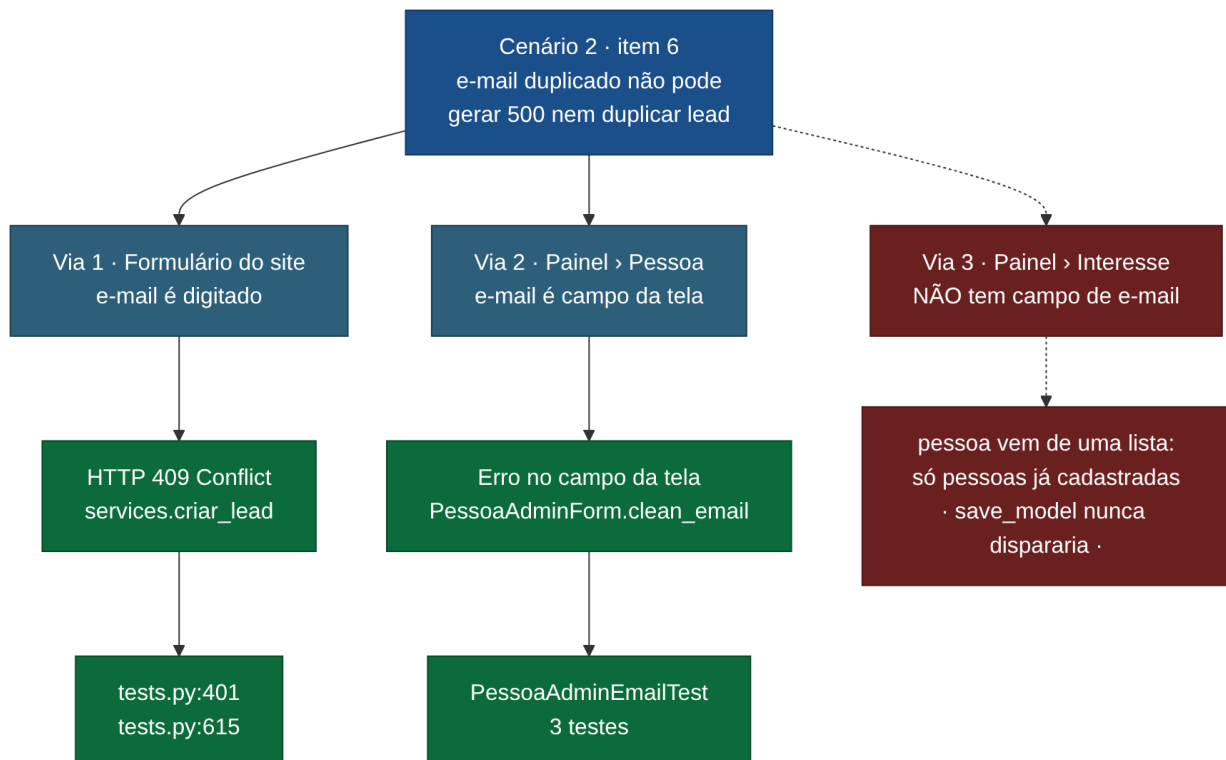


Figura 1 — Os três caminhos por onde um e-mail entra no sistema, e como cada um trata a duplicidade.

Onde o e-mail duplicado é barrado hoje:

Onde se digita o e-mail	O que trata	O que o usuário vê	Teste
Formulário do site (<code>POST /api/leads</code>)	<code>services.criar_lead</code>	Erro 409 (conflito)	<code>tests.py:401</code> (e-mail igual) <code>tests.py:615</code> (maiúsculas diferentes)
Painel > Pessoa	<code>PessoaAdminForm.clean_email</code> (<code>admin.py:37</code>)	Mensagem de erro no campo	<code>PessoaAdminEmailTest</code> (3 testes)
Painel > Interesse	—	Não se aplica: essa tela não tem campo de e-mail	

O tratamento na tela de Pessoa corrige um erro real. O sistema converte o e-mail para minúsculas na hora de salvar, mas a checagem de duplicidade acontecia **antes** dessa conversão. Resultado: cadastrar `DUPLICADO@Email.com` quando já existia `duplicado@email.com` passava pela validação e quebrava na gravação, mostrando um erro 500 ao operador. O teste `test_email_com_outra_capitalizacao_exibe_erro_e_nao_quebra` existe para garantir que isso não volte a acontecer.

O que continua diferente do documento: o lugar onde a verificação foi feita e o texto exato da mensagem. Os dois descrevem uma tela que não é a do painel do Django.

5.2. D2 — Os nomes dos status do lead

Especificação (item 8)	Sistema	Mapeamento no seed
<code>novo</code>	<code>novo</code>	direto
<code>contato</code>	<code>contatado</code>	<code>contato</code> → <code>contatado</code>
<code>qualificado</code>	<code>qualificado</code>	direto
<code>fechado</code>	<code>convertido</code>	<code>fechado</code> → <code>convertido</code>
—	<code>negociacao</code> , <code>perdido</code>	exclusivos do sistema

O sistema tem 6 status, o documento tem 4 (`models.py:223-230`). Os dois status a mais separam um lead ainda em negociação de um lead que se perdeu. O documento junta os dois casos em `fechado` , que não deixa claro se o negócio foi ganho ou perdido.

A conversão está escrita em `seed.py:69-72` e conferida pelo teste `test_status_mapeados_para_o_funil_do_sistema` .

5.3. D3 — Dígito verificador dos CPFs e CNPJs de teste

Os CPFs e CNPJs do item 8 (`111.222.333-44` , `12.345.678/0001-90` e os demais) são números de exemplo: eles não passam na validação do sistema, que confere o dígito verificador pela regra da Receita Federal.

Para que a massa de teste pudesse ser carregada, o seed **manteve o começo do número e recalculou só os dois dígitos finais**. Assim cada registro continua reconhecível:

Registro	Especificação	Seed
João da Silva	<code>111.222.333-44</code>	<code>111.222.333-96</code>
Clínica Reabilitar Lagarto	<code>12.345.678/0001-90</code>	<code>12.345.678/0001-95</code>

A outra saída seria desligar a validação de CPF e CNPJ para aceitar os números do documento — ou seja, abrir mão de uma regra real do sistema por causa de dados de teste. Conferido pelos testes `test_documentos_da_massa_passam_nos_validadores` e `test_prefixo_do_documento_e_preservado`.

5.4. D4 — A coluna `tipo_pessoa` não existe no banco (item 9.1)

O item 9.1 pede que os nomes usados no documento batam com os nomes do banco. Dos três nomes citados, **dois já estão certos** e só um é diferente — e essa diferença vem de uma decisão tomada antes deste documento:

Nome no documento	Onde ele aparece	Situação
<code>dados_fisica_cpf</code>	no envio de dados da API	existe — <code>serializers.py:48</code>
<code>dados_juridica_cnpj</code>	no envio de dados da API	existe — <code>serializers.py:49</code>
<code>peessoa_tipo_pessoa</code> (item 3.2)	—	não existe como coluna no banco

Os dois primeiros estão corretos. Os nomes `dados_fisica` e `dados_juridica` são exatamente os que o documento pede, e existem no lugar onde o próprio documento os coloca: no envio de dados da API. A ligação entre esses nomes e os do banco está escrita no código:

```
dados_fisica = PessoaFisicaReadSerializer(source='peessoa_fisica', read_only=True)
dados_juridica = PessoaJuridicaReadSerializer(source='peessoa_juridica', read_only=True)
```

São dois vocabulários: `dados_fisica` é o nome usado na API, `peessoa_fisica` é o nome usado no banco, e o `source=` liga um ao outro. Não há nada a corrigir.

O terceiro é uma decisão de modelagem, registrada em `models.py:10-26`. Guardar no banco uma coluna dizendo se a pessoa é física ou jurídica foi proposto e descartado na revisão do DER: essa informação já está no cadastro (a pessoa tem ficha de PF ou de PJ), e uma coluna extra poderia ficar desatualizada — marcada como "PF" sem que exista a ficha correspondente. Por isso o tipo é sempre consultado a partir do cadastro.

E não é só uma decisão nossa: o contrato da API em vigor já diz isso com todas as letras. O trecho abaixo é do Contrato v4.0, seção 2 — um documento aprovado *depois* da Especificação do Painel:

`tipo_pessoa` existe apenas dentro desse envio de dados (o "payload"). Ele não é guardado como uma coluna na tabela de Pessoa do banco. Serve só de instrução de "qual caminho seguir" — depois de validado, esse valor é descartado. O que realmente diferencia uma pessoa física de uma jurídica no banco é a existência de um registro relacionado na tabela `PessoaFisica` ou na tabela `PessoaJuridica`. Por isso esse campo não aparece no DER.

Atenção para não confundir: `tipo_pessoa` **existe sim no envio de dados da API**, para indicar qual conjunto de campos validar, e é descartado depois disso. O que não existe é a coluna no banco — e é por isso que ela também não aparece no DER.

A ideia original de criar essa coluna era evitar uma consulta ao banco por linha da tabela. Isso foi resolvido de outro jeito, carregando os dados de uma vez só (`admin.py:257`). Como consequência, o filtro do item 3.2 foi feito com um filtro próprio (`admin.py:60-80`) em vez do filtro automático do Django — para quem usa o painel, funciona igual.

Resumo para a homologação: os nomes que o item 9.1 pede existem na API. A única coisa que não existe é a coluna `tipo_pessoa` no banco, e isso foi uma decisão registrada do DER, confirmada depois pelo contrato em vigor.

5.5. D5 — Os nomes usados no campo de busca (itens 3.3 e 9.1)

O que a Especificação pede. O item 3.3 manda a busca procurar por `pessoa_dados_juridica_cnpj` e `pessoa_dados_fisica_cpf`. O item 9.1 pede que esses nomes batam exatamente com os do banco.

O que existe. A busca funciona por CPF e por CNPJ, como pedido, mas escrita assim (`admin.py:199`):

```
'pessoa__pessoa_fisica_cpf', 'pessoa__pessoa_juridica_cnpj'
```

Por que os nomes do documento não funcionariam aqui. Este é o mesmo assunto da seção 5.4, visto de outro ângulo. O sistema tem dois vocabulários para as mesmas tabelas:

Vocabulário	Nomes	Onde vale
Da API	<code>dados_fisica</code> , <code>dados_juridica</code>	No envio e na resposta em JSON
Do banco	<code>pessoa_fisica</code> , <code>pessoa_juridica</code>	Nas consultas ao banco — <code>models.py:85</code> e <code>:148</code>

O campo de busca do painel consulta o banco. Então ele precisa dos nomes do banco: escrito com os nomes da API, o Django não encontra o caminho e a tela sai do ar. E isso não é efeito da troca de contrato — os nomes `dados_fisica` e `dados_juridica` nunca existiram no banco, em nenhuma versão. O documento aplicou nomes de JSON num lugar que só aceita nomes de tabela.

O objetivo do item foi cumprido: quem digita um CPF ou um CNPJ na busca encontra o lead. O que mudou foi só a forma de escrever o caminho, porque a forma do documento não é executável.

Se a mesa preferir que os nomes do documento passem a funcionar também nas consultas, dá para fazer sem mexer no banco nem gerar migração: basta declarar um apelido (`related_query_name`) nos dois modelos. É uma alteração pequena e sem risco — fica à escolha de vocês.

5.6. D6 — Como a acentuação do CSV foi escrita (item 4)

O que a Especificação pede. Gerar o arquivo com a codificação `utf-8-sig`, para que o Excel abra os acentos corretamente.

O que existe. O arquivo é gerado como `utf-8` e a marca que o Excel procura (o chamado BOM) é escrita uma única vez, na primeira linha (`admin.py:322-324`).

Por quê. O Django grava o CSV em pedaços, uma linha por vez. Com `utf-8-sig`, essa marca seria repetida a cada linha gravada, e todas elas apareceriam sujas na primeira coluna da planilha. Escrevendo a marca uma vez só, o resultado no Excel é exatamente o que o item 4 quer — acentos corretos, sem lixo.

O restante do item 4 segue o documento à risca: mesmo nome de arquivo, mesmo cabeçalho de colunas e o ponto e vírgula como separador. Coberto por `InteresseAdminExportTest`.

6. Quais testes cobrem cada item

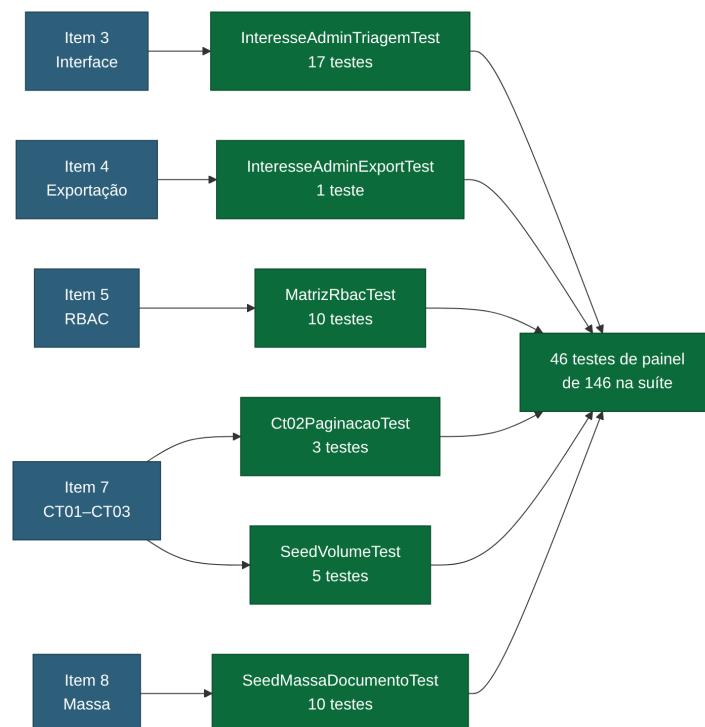


Figura 2 — Cada item da Especificação e os testes que o cobrem.

Fora esses, há mais 3 testes em `PessoaAdminEmailTest`, que cobrem o e-mail duplicado explicado na seção 5.1.

7. Como reproduzir

Para rodar todos os testes:

```
cd backend/prototipo_backend
source .venv/bin/activate
python manage.py test EasyRide
```

No fim deve aparecer `Ran 146 tests` e `OK`.

Para testar o CT02, que precisa de pelo menos 100 registros:

```
python manage.py migrate
python manage.py seed --bulk 100
python manage.py runserver
```

Os grupos de permissão são criados sozinhos durante o `migrate`. Se alguém alterar um grupo na mão pelo painel, este comando devolve tudo ao que foi combinado:

```
python manage.py sincronizar_grupos
```

8. Documentos relacionados

Documento	Local
DER (completo e simplificado)	<code>docs/der/</code>
Dicionário de Dados	<code>docs/dicionario-dados/</code>
Contrato da API v4.0 — é o que vale hoje	<code>docs/Atualizacao_Contrato_API.pdf</code>
Contrato da API v3.0 — versão anterior, para consulta	<code>docs/Especificacao_Tecnica_Contrato_API_v3.0.pdf</code>
Guia de Testes	<code>backend/prototipo_backend/TESTING.md</code>
Setup e acesso ao painel	<code>backend/prototipo_backend/README.md</code>
Versão em Markdown deste relatório	<code>docs/Conformidade_Painel_Administrativo.md</code>
Fontes de geração deste PDF	<code>docs/fontes-conformidade/</code>